Monte Carlo Simulation in Maritime Operations

An Analysis of Fuel Consumption and CO_2 Emissions

Achini Eranga Nanayakkara

Jan 14, 2025

Contents

1 Introduction **3**

2 Dataset Description **3**

3 Methodology **4**
3.1 Monte Carlo Simulation 4
3.2 Exploratory Data Analysis 4

4 Findings **4**
4.1 Correlation Analysis 4
4.2 Distribution Analysis 4
4.3 Simulation Results 6
4.4 Comparison of Actual and Simulated Data 6
4.5 Ship type, Weather and Fuel Type Analysis 6

5 Discussion **8**

6 Conclusion and Future findings **8**

A Python Code **8**

Abstract

This report presents a Monte Carlo simulation approach to analyze maritime operations, focusing on ship types, fuel consumption, and carbon dioxide emissions. The dataset used in this study includes data on various ship types, fuel consumption, and CO_2 emissions. Through the simulation, realistic maritime operations in Nigeria are modeled, incorporating variability in fuel usage patterns and emissions. The results provide valuable insights into the impact of ship types, weather conditions, and fuel types on fuel consumption and emissions, offering a basis for decision-making and predictions for policy making.

1 Introduction

Maritime operations are a vital component of transportation and trade, especially in regions like Nigeria, where waterways play a significant role. This study uses Monte Carlo simulations to assess maritime operations, focusing on fuel consumption and CO_2 emissions across various ship types. Monte Carlo methods are highly effective in simulating complex, real-world systems by accounting for uncertainty and variability. This approach allows for a deeper understanding of operational patterns and their environmental impact.

2 Dataset Description

The dataset used in this study includes the following variables:

- **Ship Type**: Categorized into Fishing Trawler, Oil Service Boat, Surfer Boat, and Tanker Ship.
- **Fuel Consumption (Liters)**: Total fuel consumed during operations.
- **CO_2 Emission (Kg)**: Amount of CO_2 emitted based on fuel consumption.
- **Weather Condition**: Categorized into calm, moderate, and stormy conditions.
- **Fuel Type**: Types of fuel used (Diesel or HFO).
- **Other Variables**: Supporting data such as engine efficiency, operated months and distance traveled.

This dataset, sourced from Kaggle (link), consists of 1,440 observations and 10 variables. The data contains no null values or extreme outliers. The variables represent common ship types, weather conditions, fuel usage patterns, and CO_2 emissions relevant to Nigerian maritime operations.

3 Methodology

3.1 Monte Carlo Simulation

Monte Carlo simulation involves generating a large number of random samples to estimate outcomes while accounting for uncertainties in input variables. In this study:

- Random samples for fuel consumption and CO_2 emissions were generated based on statistical distributions derived from historical data.
- Maritime operations were simulated under varying conditions of ship type, fuel type, and weather to estimate variability in fuel consumption and emissions.

3.2 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was conducted to identify trends and relationships among ship types, weather conditions, fuel types, fuel consumption, and CO_2 emissions. Visualization tools such as pair plots, histograms, line graphs, and correlation matrices were used to highlight key insights. Python was used for the Data analysis of this study.

4 Findings

4.1 Correlation Analysis

The correlation matrix (Table 1) indicates strong positive relationships between distance, CO_2 emissions, and fuel consumption. A weak negative correlation exists between engine efficiency and other variables, suggesting that higher engine efficiency marginally reduces emissions and fuel usage.

	Distance	Fuel Consumption	CO_2 Emission	Engine Efficiency
Distance	1	0.9451	0.9427	-0.0261
Fuel Consumption	0.9451	1	0.9972	-0.0313
CO_2 Emission	0.9427	0.9972	1	-0.0302
Engine Efficiency	-0.0261	-0.0313	-0.0302	1

Table 1: Correlation between Numerical Variables

The above correlation is visualized using Pairplot in figure (1).

4.2 Distribution Analysis

Using Maximum Likelihood Estimation (MLE) and the negative log-likelihood function, it was determined that fuel consumption and CO_2 emissions follow an exponential distribution. This finding aligns with the nature of random variations in operational data. The plot of simulation results for CO2 emission and fuel consumptions using exponential distribution visualized in figures (4a) and (4b).

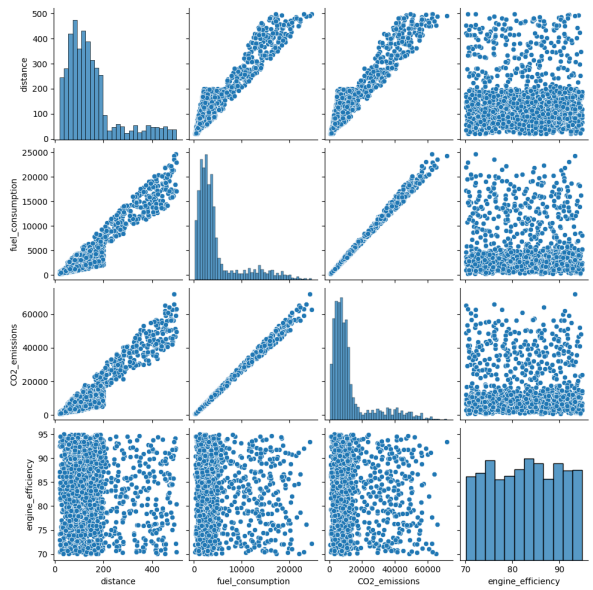
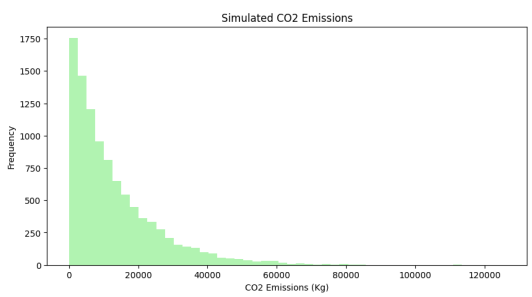
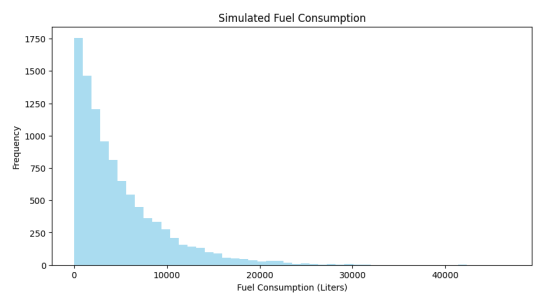


Figure 1: Comparison of numerical variables in the dataset

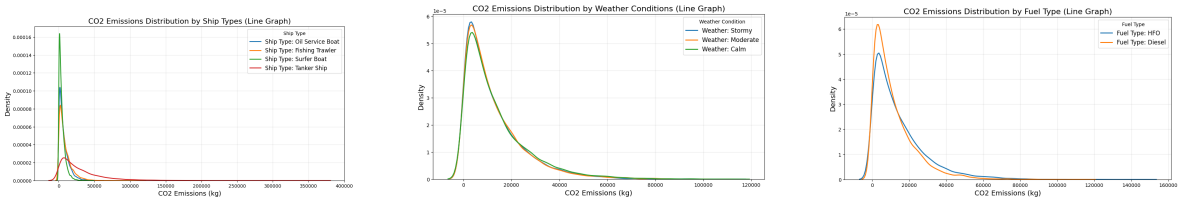


(a) CO2 emission simulation



(b) Fuel consumption simulation

Figure 2: Simulation Plots



(a) ship types (b) weather condition (c) fuel types

Figure 3: Simulated CO_2 emission for categorical variables

4.3 Simulation Results

Simulation results (Table 2) show that the mean fuel consumption and CO_2 emissions are comparable to actual data. The simulated results confirm the presence of variability while maintaining overall consistency with real-world observations.

	Mean	Std. Deviation	95% Confidence Interval
Fuel Consumption (L)	4868.03	4861.82	114.67 - 18010.19
CO_2 Emission (Kg)	13046.31	13029.69	307.33 - 48267.32

Table 2: Simulation Results for CO_2 Emissions and Fuel Consumption

Additionally, Line graphs to understand the variation of CO_2 emission with different categorical variables, ship type, whether conditions and fuel type are visualized in figure 3.

4.4 Comparison of Actual and Simulated Data

Table 3 shows that simulated fuel consumption and CO_2 emissions closely match actual data for all ship types, validating the accuracy of the simulation model.

Ship Type	Fuel Consumption (L)		CO2 Emissions (kg)	
	Actual	Simulated	Actual	Simulated
Fishing Trawler	3229.17	3225.70	8913.09	8837.78
Oil Service Boat	2619.43	2646.37	7187.83	7222.09
Surfer Boat	1666.13	1673.99	4577.63	4646.20
Tanker Ship	10780.40	10784.84	29795.43	29342.62

Table 3: Comparison of Actual and Simulated Fuel Consumption and CO2 Emissions by Ship Type

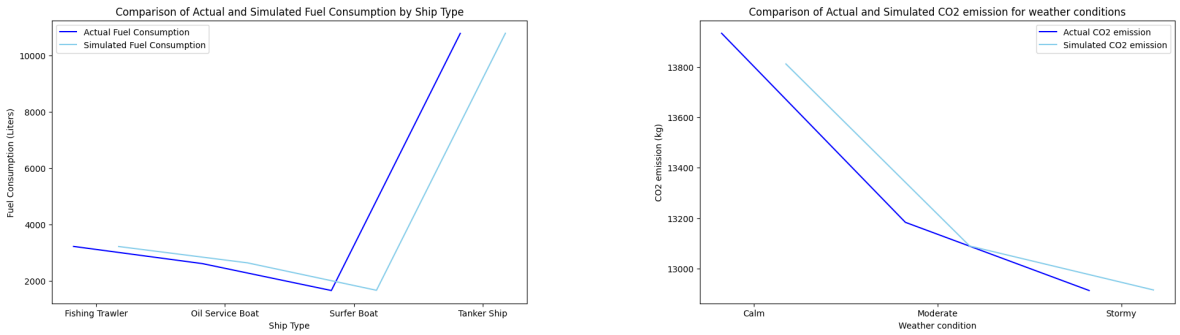
Similarly the simulation results with weather conditions shows in table 4.

4.5 Ship type, Weather and Fuel Type Analysis

Figures 4b and 5 reveal that stormy weather results in lower emissions, while Diesel fuel emits less CO_2 compared to HFO. At the same time the tanker ships consume more fuel than other ship types which may leads to higher CO_2 emission as they are

Weather Condition	Fuel Consumption (L)		CO2 Emissions (kg)	
	Actual	Simulated	Actual	Simulated
Calm	5043.07	13933.16	5070.03	13811.46
Moderate	4793.03	13183.59	4808.61	13089.67
Stormy	4673.38	12913.24	4651.22	12915.39

Table 4: Comparison of Actual and Simulated Emission and Fuel consumption by weather condition



(a) ship types (b) different weather

Figure 4: Actual vs Simulated Fuel Consumption for ship type and weather condition

highly correlated. This information is valuable for optimizing fuel choices, ship types and routing in adverse weather.

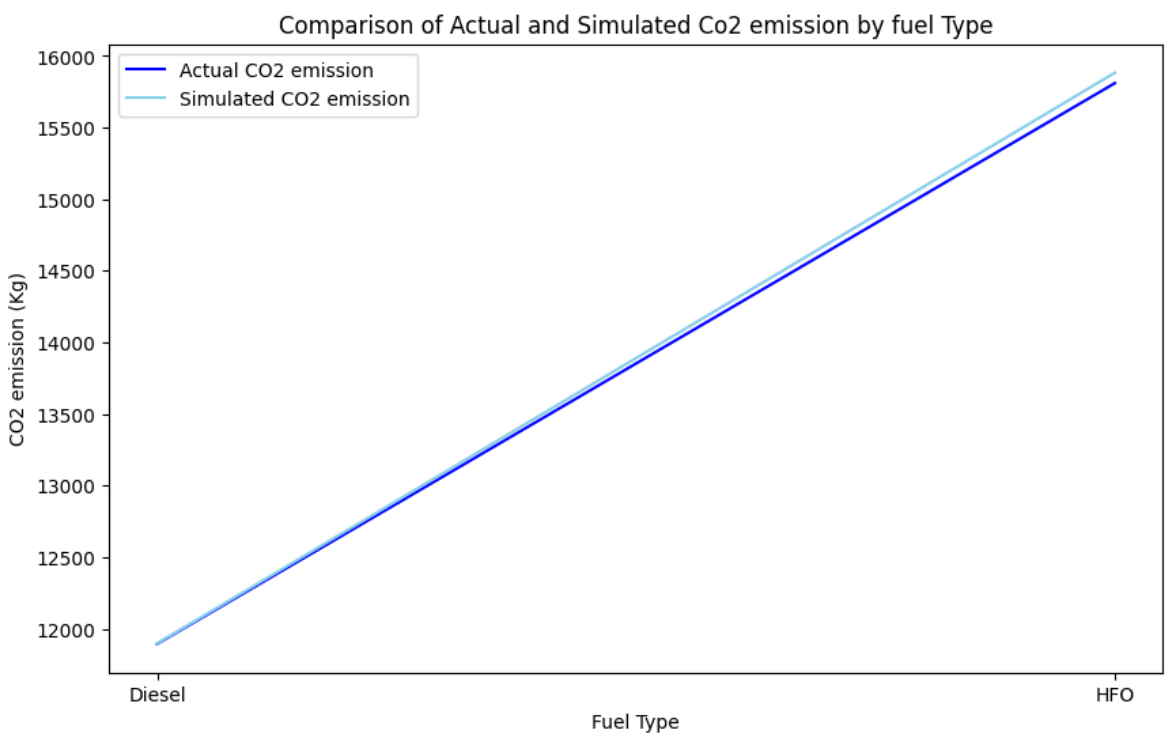


Figure 5: Comparison of Actual and simulated CO2 emission for fuel types

5 Discussion

The findings underscore the significant influence of ship type, weather, and fuel type on maritime fuel consumption and CO_2 emissions. Tanker ships exhibit the highest fuel consumption and emissions due to their size and operational demands. Weather conditions, particularly stormy weather, reduces fuel consumption and emissions which is different from expected outcomes. Additionally, the choice of fuel type plays a role, with HFO showing higher emissions compared to Diesel.

6 Conclusion and Future findings

Monte Carlo simulations effectively capture the variability in maritime operations, providing a robust framework for analyzing fuel consumption and CO_2 emissions. The study highlights the critical role of operational factors such as ship type, weather, and fuel choice in influencing environmental impacts. These insights can guide the development of sustainable practices and policies for maritime operations in Nigeria.

In future studies, sensitivity analysis can be conducted to understand how these variables differ, and seasonal effects or patterns can be further identified and explored.

A Python Code

Below is the Python code used for the analysis:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

#Load the dataset
df=pd.read_csv(r"C:\D\achini\computer intensive statistics\
    ship_fuel_efficiency.csv")

#print the first five rows of the dataset
#print(df.head())

#print descriptive statistics for the dataset
#print(df.describe())

#draw pairplot to identify the correlation among the variables
sns.pairplot(df)
plt.show()


#get the correlation for numerical variables
df[['distance', 'fuel_consumption', 'CO2_emissions','
    engine_efficiency']].corr()

```

```python
#print the MLE value to identify the appropriate distribution for numerical variables
import scipy.stats as st

for var in ['distance','CO2_emissions','fuel_consumption'] :
    distributions = [st.laplace, st.norm , st.expon]

    mles = []

    for distribution in distributions :
        pars = distribution.fit(df[var])
        mle = distribution.nnlf(pars,df[var])
        mles.append(mle)
    best_fit = sorted(zip(distributions , mles), key= lambda d: d[1])[0]
    print(f"best fit{best_fit[0].name},  mle value : {best_fit[1]} , for vaiable {var}")
#calculate lambda rate for each feature
fuel_lambda = 1/df['fuel_consumption'].mean()
emission_factor = 2.68
distance_lambda = 1/df['distance'].mean()

#numbe of simulations
num_simulations = 10000

#simulate data using exponential distributions
simulated_distance = np.random.exponential(scale = 1/distance_lambda , size = num_simulations)
simulated_fuel = np.random.exponential(scale = 1/fuel_lambda , size = num_simulations )
simulated_emissions = simulated_fuel * emission_factor


#analyse simulation results
print('\n Distance Simulation:')
print(f'mean: {np.mean(simulated_distance): .2f}km')
print(f'standard deviation: {np.std(simulated_distance): .2f}km')

# Fuel consumption statistics
print("\nFuel Consumption Simulation:")
print(f"Mean: {np.mean(simulated_fuel):.2f} liters")
print(f"Standard Deviation: {np.std(simulated_fuel):.2f} liters")

# CO2 emissions statistics
print("\nCO2 Emission Simulation:")
print(f"Mean: {np.mean(simulated_emissions):.2f} kg")
print(f"Standard Deviation: {np.std(simulated_emissions):.2f} kg")
```

```python

# Visualization of Results

# Plot simulated fuel consumption histogram
plt.figure(figsize=(10, 5))
plt.hist(simulated_fuel, bins=50, color='skyblue', alpha=0.7)
plt.title("Simulated Fuel Consumption")
plt.xlabel("Fuel Consumption (Liters)")
plt.ylabel("Frequency")
plt.show()

# Plot simulated CO2 emissions histogram
plt.figure(figsize=(10, 5))
plt.hist(simulated_emissions, bins=50, color='lightgreen', alpha=0.7)
plt.title("Simulated CO2 Emissions")
plt.xlabel("CO2 Emissions (Kg)")
plt.ylabel("Frequency")
plt.show()

# Confidence Intervals
fuel_lower, fuel_upper = np.percentile(simulated_fuel, [2.5, 97.5])
emissions_lower, emissions_upper = np.percentile(simulated_emissions, [2.5, 97.5])

print(f"\nFuel Consumption 95% Confidence Interval: {fuel_lower:.2f} - {fuel_upper:.2f} liters")
print(f"CO2 Emissions 95% Confidence Interval: {emissions_lower:.2f} - {emissions_upper:.2f} kg")

# Analyze by categorical data
categories = df['ship_type'].unique()
results = {}

for category in categories:
    # Subset data for the current category
    subset = df[df['ship_type'] == category]

    # Calculate exponential lambda for fuel consumption and distance
    fuel_lambda = 1 / subset['fuel_consumption'].mean()
    distance_lambda = 1 / subset['distance'].mean()

    # Monte Carlo Simulation for this category
    simulated_distance = np.random.exponential(scale=1/distance_lambda, size=num_simulations)
    simulated_fuel = np.random.exponential(scale=1/fuel_lambda, size=num_simulations)
    simulated_emissions = simulated_fuel * emission_factor
```

```python

    # Store results
    results[category] = {
        "mean_fuel": np.mean(simulated_fuel),
        "mean_emissions": np.mean(simulated_emissions),
        "distance_ci": np.percentile(simulated_distance, [2.5, 97.5]),
        "fuel_ci": np.percentile(simulated_fuel, [2.5, 97.5]),
        "emissions_ci": np.percentile(simulated_emissions, [2.5, 97.5]),
    }



plt.bar(df['weather_conditions'], df['CO2_emissions'])
plt.bar(df['fuel_type'], df['CO2_emissions'])
plt.title('CO2 emission of the dataset according to weather condition and fuel type')
plt.show()
plt.bar(df['ship_type'], df['CO2_emissions'])
plt.title('CO2 emission according to ship_type')
plt.show()

plt.bar(df['month'],df['CO2_emissions'])
labels =['jan','feb','mar','apr','may','jun','jul','aug','sep','oct','nov','dec']
plt.xticks(range(0, 12), labels)
plt.title('Comparison of CO2 emission by month')
plt.show()
# Line graph for CO2 emissions by ship types
plt.figure(figsize=(12, 6))

for ship_type, metrics in results.items():
    # Simulated CO2 emissions for this ship type
    simulated_emissions = np.random.exponential(
        scale=1/(1/metrics['mean_emissions']), size=num_simulations
    )
    # Plot the density curve
    sns.kdeplot(
        simulated_emissions,
        label=f"Ship Type: {ship_type}",
        linewidth=2
    )

# Add titles and labels
plt.title("CO2 Emissions Distribution by Ship Types (Line Graph)", fontsize=16)
plt.xlabel("CO2 Emissions (kg)", fontsize=14)
plt.ylabel("Density", fontsize=14)
```

```python
plt.legend(title="Ship Type", fontsize=12)
plt.grid(alpha=0.3)
plt.show()

# Unique weather conditions in the dataset
weather_conditions = df['weather_conditions'].unique()
weather_results = {}

# Number of simulations
num_simulations = 10000

for weather in weather_conditions:
    # Subset the dataset for the current weather condition
    subset = df[df['weather_conditions'] == weather]

    # Calculate lambda (rate) for fuel consumption and distance
    fuel_lambda = 1 / subset['fuel_consumption'].mean()
    distance_lambda = 1 / subset['distance'].mean()

    # Perform Monte Carlo simulations for fuel, distance, and CO2 emissions
    simulated_distance = np.random.exponential(scale=1/distance_lambda, size=num_simulations)
    simulated_fuel = np.random.exponential(scale=1/fuel_lambda, size=num_simulations)
    simulated_emissions = simulated_fuel * emission_factor

    # Store simulation results for this weather condition
    weather_results[weather] = {
        "mean_fuel": np.mean(simulated_fuel),
        "mean_emissions": np.mean(simulated_emissions),
        "distance_ci": np.percentile(simulated_distance, [2.5, 97.5]),
        "fuel_ci": np.percentile(simulated_fuel, [2.5, 97.5]),
        "emissions_ci": np.percentile(simulated_emissions, [2.5, 97.5]),
    }


# Line graph for CO2 emissions by weather conditions
plt.figure(figsize=(12, 6))

for weather, metrics in weather_results.items():
    # Simulated CO2 emissions for this weather condition
    simulated_emissions = np.random.exponential(
        scale=1/(1/metrics['mean_emissions']), size=num_simulations
    )
    # Plot the density curve
    sns.kdeplot(
```

```
        simulated_emissions,
        label=f"Weather: {weather}",
        linewidth=2
    )

# Add titles and labels
plt.title("CO2 Emissions Distribution by Weather Conditions (Line Graph)", fontsize=16)
plt.xlabel("CO2 Emissions (kg)", fontsize=14)
plt.ylabel("Density", fontsize=14)
plt.legend(title="Weather Condition", fontsize=12)
plt.grid(alpha=0.3)
plt.show()


# Unique fuel types in the dataset
fuel_types = df['fuel_type'].unique()
fuel_type_results = {}

# Number of simulations
num_simulations = 10000

for fuel in fuel_types:
    # Subset the dataset for the current fuel type
    subset = df[df['fuel_type'] == fuel]

    # Calculate lambda (rate) for fuel consumption and distance
    fuel_lambda = 1 / subset['fuel_consumption'].mean()
    distance_lambda = 1 / subset['distance'].mean()

    # Use specific emission factors for different fuel types (example values)
    if fuel == "Diesel":
        emission_factor = 2.68  # kg CO2/L
    elif fuel == "LNG":
        emission_factor = 2.75  # kg CO2/L
    elif fuel == "Heavy Fuel Oil":
        emission_factor = 3.15  # kg CO2/L
    else:
        emission_factor = 2.5  # Default value

    # Perform Monte Carlo simulations for fuel, distance, and CO2 emissions
    simulated_distance = np.random.exponential(scale=1/distance_lambda, size=num_simulations)
    simulated_fuel = np.random.exponential(scale=1/fuel_lambda, size=num_simulations)
    simulated_emissions = simulated_fuel * emission_factor

    # Store simulation results for this fuel type
```

```
    fuel_type_results[fuel] = {
        "mean_fuel": np.mean(simulated_fuel),
        "mean_emissions": np.mean(simulated_emissions),
        "distance_ci": np.percentile(simulated_distance, [2.5, 97.5]),
        "fuel_ci": np.percentile(simulated_fuel, [2.5, 97.5]),
        "emissions_ci": np.percentile(simulated_emissions, [2.5, 97.5]),
    }


import seaborn as sns

# Prepare data for line graph
plt.figure(figsize=(12, 6))

for fuel, metrics in fuel_type_results.items():
    # Simulated CO2 emissions for this fuel type
    simulated_emissions = np.random.exponential(
        scale=1/(1/metrics['mean_emissions']), size=num_simulations
    )
    # Plot the density curve
    sns.kdeplot(
        simulated_emissions,
        label=f"Fuel Type: {fuel}",
        linewidth=2
    )

# Add titles and labels
plt.title("CO2 Emissions Distribution by Fuel Type (Line Graph)", fontsize=16)
plt.xlabel("CO2 Emissions (kg)", fontsize=14)
plt.ylabel("Density", fontsize=14)
plt.legend(title="Fuel Type", fontsize=12)
plt.grid(alpha=0.3)
plt.show()

# Monte Carlo Simulation for Ship Type
ship_types = df['ship_type'].unique()
simulation_results_ship_type = []

for ship_type in ship_types:
    subset = df[df['ship_type'] == ship_type]

    # Simulate fuel consumption and CO2 emissions using exponential distribution
    fuel_consumption_sim = np.random.exponential(scale=subset['fuel_consumption'].mean(), size=10000)
    co2_emissions_sim = np.random.exponential(scale=subset['
```

```
        CO2_emissions'].mean(), size=10000)

    simulation_results_ship_type.append((ship_type, fuel_consumption_sim.mean(), co2_emissions_sim.mean()))



# Compare with actual data
ship_type_comparison = pd.DataFrame(simulation_results_ship_type, columns=['ship_type', 'Simulated Fuel Consumption', 'Simulated CO2 Emissions'])
ship_type_actual = df.groupby('ship_type')[['fuel_consumption', 'CO2_emissions']].mean().reset_index()
comparison_ship_type = pd.merge(ship_type_actual, ship_type_comparison, on = 'ship_type', suffixes=('_Actual', '_Simulated'))

print(comparison_ship_type)


# Plotting Comparison for Ship Type
plt.figure(figsize=(10, 6))
x = np.arange(len(comparison_ship_type['ship_type']))
width = 0.35

plt.plot(x - width/2, comparison_ship_type['fuel_consumption'], label='Actual Fuel Consumption', color='blue')
plt.plot(x + width/2, comparison_ship_type['Simulated Fuel Consumption'], label='Simulated Fuel Consumption', color='skyblue')
plt.xticks(x, comparison_ship_type['ship_type'])
plt.xlabel('Ship Type')
plt.ylabel('Fuel Consumption (Liters)')
plt.title('Comparison of Actual and Simulated Fuel Consumption by Ship Type')
plt.legend()
plt.show()


weather_conditions = df['weather_conditions'].unique()
simulation_results_weather = []

for weather in weather_conditions:
    subset = df[df['weather_conditions'] == weather]

    # Simulate fuel consumption and CO2 emissions
    fuel_consumption_sim = np.random.exponential(subset['fuel_consumption'].mean(), size=10000)
    co2_emissions_sim = np.random.exponential(subset['CO2_emissions'].mean(), size=10000)
```

```python

    simulation_results_weather.append((weather, fuel_consumption_sim.mean(), co2_emissions_sim.mean()))

# Create a comparison DataFrame
weather_comparison = pd.DataFrame(simulation_results_weather, columns=['weather_conditions', 'Simulated Fuel Consumption', 'Simulated CO2 Emissions'])
weather_actual = df.groupby('weather_conditions')[['fuel_consumption', 'CO2_emissions']].mean().reset_index()
comparison_weather = pd.merge(weather_actual, weather_comparison, on='weather_conditions', suffixes=('_Actual', '_Simulated'))

print(comparison_weather)

# Plotting Comparison for weather Type
plt.figure(figsize=(10, 6))
x = np.arange(len(comparison_weather['weather_conditions']))
width = 0.35

plt.plot(x - width/2, comparison_weather['CO2_emissions'], label='Actual CO2 emission', color='blue')
plt.plot(x + width/2, comparison_weather['Simulated CO2 Emissions'], label='Simulated CO2 emission', color='skyblue')
plt.xticks(x, comparison_weather['weather_conditions'])
plt.xlabel('Weather condition')
plt.ylabel('CO2 emission (kg)')
plt.title('Comparison of Actual and Simulated CO2 emission for weather conditions')
plt.legend()
plt.show()
fuel_types = df['fuel_type'].unique()
simulation_results_fuel = []

for fuel in fuel_types:
    subset = df[df['fuel_type'] == fuel]

    # Simulate CO2 emissions
    co2_emissions_sim = np.random.exponential(scale=subset['CO2_emissions'].mean(), size=10000)

    simulation_results_fuel.append((fuel, co2_emissions_sim.mean()))

# Create a comparison DataFrame
fuel_comparison = pd.DataFrame(simulation_results_fuel, columns=['fuel_type', 'Simulated CO2 Emissions'])
fuel_actual = df.groupby('fuel_type')[['CO2_emissions']].mean().reset_index()
```

```
comparison_fuel = pd.merge(fuel_actual, fuel_comparison, on='fuel_type', suffixes=('_Actual', '_Simulated'))

print(comparison_fuel)

# Plotting Comparison for fuel Type
plt.figure(figsize=(10, 6))
x = np.arange(len(comparison_fuel['fuel_type']))
width = 0.35

plt.plot(x , comparison_fuel['CO2_emissions'], label='Actual CO2 emission', color='blue')
plt.plot(x , comparison_fuel['Simulated CO2 Emissions'], label='Simulated CO2 emission', color='skyblue')
plt.xticks(x, comparison_fuel['fuel_type'])
plt.xlabel('Fuel Type')
plt.ylabel('CO2 emission (Kg)')
plt.title('Comparison of Actual and Simulated Co2 emission by fuel Type')
plt.legend()
plt.show()
```

Listing 1: Monte Carlo Simulation Code